VINUNIVERSITY

Deployment — Đưa Agent Lên Cloud

AICB-P1 · Ngày 12 · Từ localhost đến production URL

Nguyễn Hải Dương

VinUniversity · Phase 1 · 2026



Nguyễn Hải Dương

- AI Tech Lead — FPT AI Center
- Visiting Lecturer — VinUniversity
- Engineer — Center for AI Research (CAIR), VinUniversity

Email: duong.nh@vinuni.edu.vn
LinkedIn: linkedin.com/in/nguyễn-hải-dương

HÃY SUY NGHĨ...

“Bạn demo cho sếp thấy agent chạy trên laptop. Sếp hỏi: khi nào 100 người dùng được?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1. Cloud infrastructure: AWS, GCP, Azure
2. Từ localhost đến production
3. Docker & containerization
4. Cloud deployment options
5. API gateway & security
6. Scaling & reliability
7. Lab 12 + deliverable
8. Preview Day 13

Mục Tiêu Ngày 12

- Đọc được **bản đồ dịch vụ** của AWS / GCP / Azure: compute, storage, networking, IAM, database, data platform, AI — và biết cái nào tương đương cái nào
- Hiểu **gap giữa dev và production**: dependencies, config, secrets, networking
- Viết **Dockerfile** đúng cách để đóng gói agent thành container
- So sánh các **cloud deployment options**: Railway, Render, Cloud Run, serverless
- Thiết kế **API gateway** với authentication, rate limiting, cost protection
- Deploy agent lên cloud và có **public URL** hoạt động được

Deliverable Cuối Ngày

Artifact pack cần nộp

Agent đã được containerize và deploy lên cloud, có health check endpoint, basic authentication, và truy cập được qua public URL

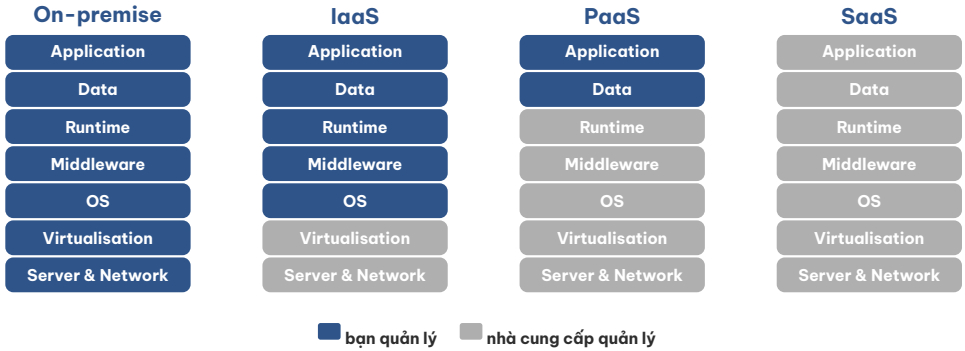
- 1 Dockerfile + docker-compose cho agent + dependencies
- 1 deployed instance trên Railway hoặc Render
- 1 health check endpoint trả về status
- 1 public URL mà bất kỳ ai cũng có thể truy cập và dùng agent

Cloud Infrastructure: AWS · GCP · Azure

Trước khi chọn chỗ deploy, cần biết trên cloud có sẵn những gì — và ba nhà cung cấp lớn gọi cùng một thứ bằng ba cái tên khác nhau

01

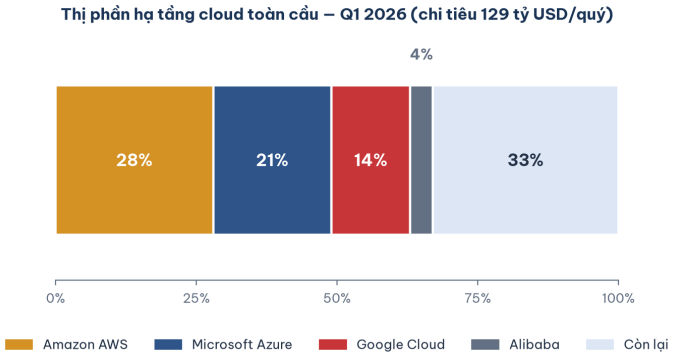
Cloud Là Gì? Thuê Thay Vì Mua



Đọc hình theo chiều ngang

Càng sang phải càng **bớt việc** và càng **mất quyền kiểm soát**. Agent của lớp này sống ở giữa: **PaaS container**.

Ba Ông Lớn Chia Nhau Hơn 60% Thị Trường



Vì sao con số này quan trọng với bạn: thị phần quyết định *hệ sinh thái* — số lượng tutorial, thư viện SDK, câu trả lời trên Stack Overflow, và số kỹ sư bạn tuyển được. Một dịch vụ tốt nhưng ít người dùng sẽ tốn của bạn nhiều giờ debug hơn. Cả ba đều thừa sức chạy agent của lớp này; khác biệt nằm ở **giá, hệ sinh thái AI, và công ty bạn đã ký hợp đồng với ai**.

Nguồn: Synergy Research Group, Q1 2026 (qua Statista chart 18819)

Ba Triết Lý Khác Nhau

Google Cloud

Data & AI first. BigQuery và Vertex AI là lý do người ta sang GCP.

Mạnh: Kubernetes (GKE), data platform, giá per-second.

Yếu: ít dịch vụ enterprise ngách hơn.

AWS

Rộng nhất. 200+ dịch vụ, ra đời sớm nhất (2006).

Mạnh: độ phủ, tài liệu, thị trường lao động.

Yếu: nhiều lựa chọn quá dễ lạc; console phức tạp.

Microsoft Azure

Enterprise & hybrid. Gắn chặt với Windows, AD, Office 365.

Mạnh: Azure OpenAI, hợp đồng doanh nghiệp, on-prem lai.

Yếu: tên dịch vụ đổi liên tục; portal nặng.

Lưu ý: Đừng chọn cloud theo cảm tính. Ba câu hỏi thực tế: **(1)** công ty đã có hợp đồng với ai? **(2)** team đã biết cái nào? **(3)** cần dịch vụ AI nào?

Compute — Nơi Code Thật Sự Chạy

Hạng mục	Google Cloud	AWS	Azure
Máy ảo (IaaS)	Compute Engine	EC2	Virtual Machines
Kubernetes quản trị	GKE	EKS	AKS
Container không cần quản server	Cloud Run	ECS + Fargate, App Runner	Container Apps
Function (FaaS)	Cloud Run functions	Lambda	Azure Functions
PaaS truyền thống	App Engine	Elastic Beanstalk	App Service
Batch / job hàng loạt	Cloud Batch	AWS Batch	Azure Batch
Chạy ở biên (edge)	Cloud Run (edge), Cloudflare *	Lambda@Edge, CloudFront Functions	Azure Front Door rules

* Cloudflare Workers không thuộc ba ông lớn nhưng là lựa chọn edge phổ biến nhất.

Tên dịch vụ ở cùng một hàng là tương đương về vai trò, không phải giống hệt về tính năng.

Storage — Chỗ Để Dữ Liệu Nằm

Hạng mục	Google Cloud	AWS	Azure
Object storage	Cloud Storage (GCS)	S3	Blob Storage
Block (ổ đĩa gắn VM)	Persistent Disk, Hyperdisk	EBS	Managed Disks
File share (NFS/SMB)	Filestore	EFS, FSx	Azure Files
Lưu trữ lạnh / archive	Archive Storage class	S3 Glacier	Archive tier
Đồng bộ từ on-prem	Storage Transfer Service	Storage Gateway, DataSync	Azure File Sync
Tầng giá	Standard → Nearline → Coldline → Archive	Standard → IA → Glacier	Hot → Cool → Cold → Archive

Object storage là mặc định cho mọi thứ không phải database: file người dùng upload, log, checkpoint model, corpus RAG.

Networking — Đưa Request Đến Đúng Chỗ

Hạng mục	Google Cloud	AWS	Azure
Mạng riêng ảo	VPC	VPC	Virtual Network (VNet)
Load balancer	Cloud Load Balancing	ELB: ALB / NLB	Load Balancer, App Gateway
DNS	Cloud DNS	Route 53	Azure DNS
CDN	Cloud CDN, Media CDN	CloudFront	Front Door, Azure CDN
API gateway	API Gateway, Apigee	API Gateway	API Management
Kết nối riêng tới on-prem	Cloud Interconnect, Private Service Connect	Direct Connect, PrivateLink	ExpressRoute, Private Link

Networking là phần dễ bị bỏ qua nhất khi học cloud — và là nguyên nhân phổ biến nhất của hoá đơn bất ngờ (egress traffic).

Security & IAM — Ai Được Làm Gì

Hạng mục	Google Cloud	AWS	Azure
Danh tính & phân quyền	Cloud IAM	IAM, IAM Identity Center	Entra ID (Azure AD)
Quản lý secret	Secret Manager	Secrets Manager, Parameter Store	Key Vault
Khoá mã hoá	Cloud KMS	KMS, CloudHSM	Key Vault, Managed HSM
WAF & chống DDoS	Cloud Armor	WAF + Shield	WAF + DDoS Protection
Phát hiện mối đe doạ	Security Command Center	GuardDuty, Security Hub	Defender for Cloud
Chứng chỉ TLS	Certificate Manager	Certificate Manager (ACM)	Key Vault certificates
Danh tính cho workload	Workload Identity Federation	IAM Roles for Service Accounts	Managed Identity

Nguyên tắc chung cho cả ba: least privilege, workload identity thay cho long-lived key, secret không bao giờ nằm trong code.

Database — SQL, NoSQL, Cache, Vector

Hạng mục	Google Cloud	AWS	Azure
SQL quản trị	Cloud SQL, AlloyDB	RDS, Aurora	Azure SQL, DB for PostgreSQL
NoSQL key-value	Firestore, Bigtable	DynamoDB	Cosmos DB
Cache trong bộ nhớ	Memorystore	ElastiCache (Redis)	Azure Cache for Redis
Vector search	AlloyDB ScaNN, Vertex AI Vector Search	OpenSearch, Aurora pgvector	AI Search, Cosmos DB
Graph	Spanner Graph	Neptune	Cosmos DB (Gremlin)
Phân tán toàn cầu	Spanner	Aurora Global, DynamoDB Global Tables	Cosmos DB

Cho agent RAG của lớp này: một Postgres quản trị có pgvector thường đủ — đừng vội mua vector database riêng.

Data Platform — Từ Log Thô Đến Bảng Phân Tích

Hạng mục	Google Cloud	AWS	Azure
Data warehouse	BigQuery	Redshift	Synapse, Microsoft Fabric
Data lake / lakehouse	Cloud Storage + BigLake	S3 + Lake Formation	Data Lake Storage Gen2
ETL / xử lý theo lô	Dataflow, Dataproc	Glue, EMR	Data Factory, Synapse Spark
Streaming & message	Pub/Sub	Kinesis, MSK (Kafka)	Event Hubs
Điều phối pipeline	Cloud Composer (Airflow)	Step Functions, MWAA (Airflow)	Data Factory, Logic Apps
BI / dashboard	Looker, Looker Studio	QuickSight	Power BI

Đây là tầng nuôi RAG: corpus được ingest, làm sạch và chunk ở đây trước khi vào vector store.

AI & ML Platform — Tầng Gần Agent Nhất

Hạng mục	Google Cloud	AWS	Azure
Nền tảng ML tổng quát	Vertex AI	SageMaker	Azure Machine Learning
LLM API quản trị	Vertex AI (Gemini)	Bedrock	Azure OpenAI, AI Foundry
RAG / knowledge base	Vertex AI Search	Bedrock Knowledge Bases	Azure AI Search
Runtime cho agent	Vertex AI Agent Engine	Bedrock AgentCore	AI Foundry Agent Service
GPU / accelerator	A3/A4 VM, Cloud TPU	EC2 P5, G6	NC, ND series
Vision / speech dựng sẵn	Vision AI, Speech-to-Text	Rekognition, Transcribe	AI Vision, AI Speech

Tên dịch vụ AI đổi nhanh hơn mọi tầng khác — luôn kiểm tra lại tài liệu chính thức trước khi viết vào kiến trúc.

Những Cái Tên Bạn Sẽ Gặp Nhiều Nhất Trên AWS


EC2 · VM


S3 · object


Lambda · FaaS


ECS · container



EKS · K8s


Fargate


RDS · SQL


DynamoDB

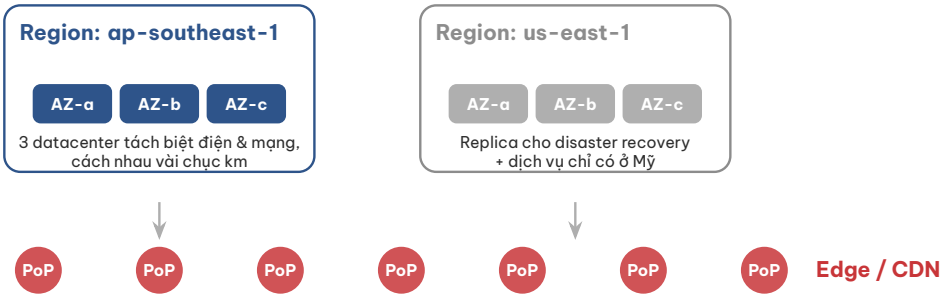

Redshift


API Gateway


IAM


CloudWatch

Region, Availability Zone, Edge — Địa Lý Của Cloud



Ba quyết định thực tế

(1) Chọn region gần người dùng — Singapore cho Việt Nam. **(2)** Trải instance qua nhiều AZ để 1 datacenter chết không làm chết agent. **(3)** Dữ liệu ra khỏi region **tốn tiền egress**; giữ agent và database cùng region.

Shared Responsibility — Cloud Lo Gì, Bạn Lo Gì

VINUNIVERSITY

Cloud lo: security OF the cloud

- Datacenter vật lý, điện, làm mát
- Phần cứng, hypervisor, mạng lõi
- Vá lỗi dịch vụ quản trị (RDS, Cloud Run…)
- Chứng chỉ tuân thủ (ISO, SOC 2)

Bạn lo: security IN the cloud

- IAM policy, ai được gọi API nào
- Secret, API key, rotation
- Cấu hình bucket public hay private
- Code, dependency, prompt injection

Lưu ý: Gần như mọi vụ rò rỉ dữ liệu trên cloud đều nằm ở cột phải: **một S3 bucket để public**, **một IAM role quá rộng**, **một API key commit lên GitHub**. Cloud không cứu bạn khỏi cấu hình sai.

Chọn Cloud Nào Cho Agent Của Bạn?

- **Đã dùng Microsoft 365 / Active Directory** → Azure. Danh tính có sẵn, Azure OpenAI ngồi cạnh dữ liệu nội bộ.
- **Bài toán nặng data / analytics** → GCP. BigQuery + Vertex AI là combo khó bị đánh bại cho pipeline RAG quy mô lớn.
- **Cần nhiều dịch vụ ngách, tuyển người dễ** → AWS. Gần như mọi thứ đều có và mọi thứ đều có người đã viết blog về nó.
- **Chỉ cần agent chạy được tuần này** → **đừng chọn cả ba**. Railway hoặc Render, 10 phút, không cần học IAM.

Free tier để học

AWS Free Tier, GCP $300 credit 90 ngày, Azure $200 credit 30 ngày. Cả ba đều **yêu cầu thẻ tín dụng** và đều có thể phát sinh phí — **đặt budget alert ngay khi tạo tài khoản**, trước cả khi deploy gì.

Từ Localhost Đến Production

Agent chạy trên máy mình khác rất xa với agent chạy cho 100 người — gap đó không chỉ là “copy code lên server”

Recap: Agent Đã Hoàn Chỉnh Nhưng Chỉ Ở Local

11 ngày đã build

- LLM API + prompt engineering
- RAG pipeline grounded
- Multi-agent + MCP
- UX + trust layer
- Guardrails + safety

Nhưng đang chạy trên

- localhost:8000
- API keys trong .env file
- Chỉ 1 user (chính mình)
- Không health check
- Tắt laptop = agent chết

Lưu ý: "It works on my machine" là câu nói nổi tiếng nhất trong lịch sử software engineering. Day 12 giải quyết đúng vấn đề này.

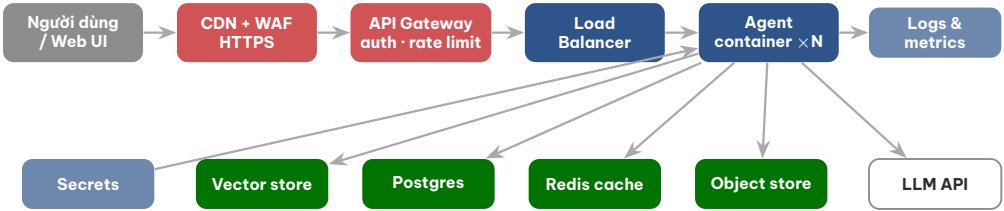
Dev Environment ≠ Production Environment

Khía cạnh	Dev (localhost)	Production
Dependencies	“pip install” thủ công	Đóng gói cùng container, phiên bản pin cứng
Config	.env file trên máy	Environment variables, secrets manager
Networking	localhost:8000	HTTPS, domain, load balancer, VPC
Users	1 (chính mình)	N users đồng thời, không tin cậy
Failure	Restart thủ công	Auto-restart, health check, alert
Observability	`print()` ra terminal	Structured log, metric, trace tập trung
Cost	Không đáng kể	Từng request đều tốn tiền token

Nguyên tắc

Environment parity: dev/staging/prod **càng giống nhau càng ít bug** khi deploy.

Kiến Trúc Tham Chiếu: Agent Trên Cloud



Đọc hình

Hàng trên là **đường đi của request**, hàng dưới là **trạng thái agent cần**. Container ở giữa phải **stateless** — mọi thứ cần nhớ đều nằm ở hàng dưới, nên thêm bớt instance không làm mất dữ liệu ai.

12-Factor App — Áp Dụng Cho AI Agent

VINUNIVERSITY

4 nguyên tắc quan trọng nhất

1. **Config in env:** không hardcode API keys
2. **Stateless processes:** agent không giữ state trên instance
3. **Port binding:** expose service qua cổng platform cấp
4. **Dev/prod parity:** giữ gap nhỏ nhất

4 cái tiếp theo khi hệ thống lớn lên

5. **Logs as streams:** ghi ra stdout, để platform thu gom
6. **Disposability:** khởi động nhanh, tắt êm (graceful shutdown)
7. **Build/release/run:** tách rõ ba giai đoạn
8. **Admin processes:** migration chạy như job riêng

Lưu ý: Đừng cố học hết 12 factors ngay. 4 cái bên trái đủ cho MVP deployment. Mở rộng dần sang cột phải khi hệ thống lớn lên.

Secrets — Từ .env Đến Secrets Manager

Hạng mục	Google Cloud	AWS	Azure
Kho secret	Secret Manager	Secrets Manager	Key Vault
Config không nhạy cảm	Runtime env vars	SSM Parameter Store	App Configuration
Tự động xoay vòng	Có (phiên bản secret)	Có (Lambda rotation)	Có (rotation policy)
App xác thực bằng gì	Service account gắn revision	IAM role gắn task	Managed identity
Tương đương ở PaaS nhỏ	Railway / Render: env vars trong dashboard — mã hoá at rest, không hiện trong log build		

Quy tắc chung: ứng dụng đọc secret lúc khởi động qua danh tính của chính nó, không cầm sẵn một API key vĩnh viễn.

Dev → Staging → Production



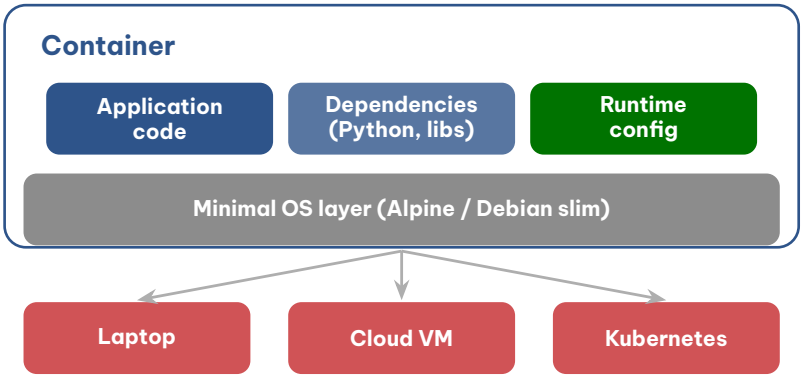
Điểm mấu chốt

Cùng **một image** đi qua cả ba môi trường — chỉ environment variables đổi. Nếu staging build một image khác production, staging không còn chứng minh được điều gì.

Docker — Đóng Gói Agent Thành Container

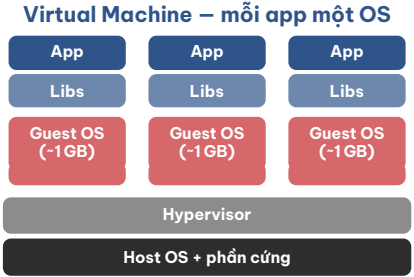
Container giải quyết “it works on my machine” bằng cách đóng gói mọi thứ agent cần thành 1 unit chạy được ở bất kỳ đâu

Container Là Gì?

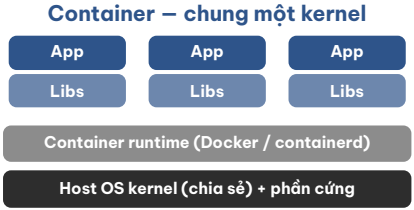


Ý chính: Container = app + deps + runtime đóng gói thành 1 unit. **Build 1 lần, chạy ở mọi nơi.**

Virtual Machine vs Container



Khởi động: **phút** · Ảnh: **GB**
Cách ly ở mức phần cứng



Khởi động: **giây** · Ảnh: **MB**
Cách ly ở mức tiến trình

Lưu ý: Container nhẹ vì **không chở theo kernel**. Đổi lại, cách ly yếu hơn VM — chạy code không tin cậy của người lạ thì vẫn cần sandbox hoặc microVM (Firecracker, gVisor).

Dockerfile Cho AI Agent

```
FROM python:3.11-slim AS builder
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
RUN useradd --create-home --uid 10001 agent
COPY --from=builder /install /usr/local
COPY --chown=agent:agent . .
USER agent
HEALTHCHECK CMD curl -fsS http://localhost:8000/health || exit 1
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Ba dòng đáng nhớ

Builder bị vứt đi — compiler không lên production. **USER agent**: không chạy root. **HEALTHCHECK**: platform biết khi nào restart.

Layer Caching — Thứ Tự Lệnh Quyết Định Tốc Độ Build

VINUNIVERSITY

Sai — rebuild 3 phút mỗi lần

```
COPY . .
RUN pip install -r requirements.txt
```

Sửa 1 dòng code → layer `COPY` đổi → `pip install` chạy lại từ đầu.

Đúng — rebuild 10 giây

```
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

Code đổi nhưng `requirements.txt` không → Docker dùng lại layer đã cache.

Quy tắc một câu

Sắp xếp Dockerfile theo **tần suất thay đổi tăng dần**: thứ ít đổi nhất (base image, system package, dependency) lên trên, code của bạn xuống dưới cùng.

Docker Compose — Multi-Service Setup

Agent stack điển hình

- **Agent:** FastAPI + LLM logic
- **Vector store:** Qdrant / Chroma
- **Cache:** Redis cho rate limit
- **Metadata:** lịch sử chat

Lợi ích

- 1 lệnh `docker compose up` khởi động toàn bộ stack
- Các service gọi nhau bằng **tên service**, không cần biết IP
- File compose chính là tài liệu kiến trúc luôn cập nhật

Cho lab

Bắt đầu với 2 services: **agent + vector store**. Thêm Redis và Nginx khi hệ thống cần scale.

docker-compose.yml Tối Thiểu Cho Agent

```yaml
services:
  agent:
    build: .
    ports: ["8000:8000"]
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}   # from the shell, never in the file
      - QDRANT_URL=http://vectordb:6333    # services reach each other by name
    depends_on:
      vectordb: {condition: service_healthy}
    restart: unless-stopped
  vectordb:
    image: qdrant/qdrant:v1.12.1           # pin the version, never :latest
    volumes: ["qdrant_data:/qdrant/storage"]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:6333/healthz"]

volumes:
  qdrant_data:
```

Image Size Optimization

Image lớn (>1GB)

- deploy chậm (push/pull lâu)
- tốn storage và bandwidth
- cold start chậm hơn
- attack surface lớn hơn

Image gọn (<500MB)

- deploy nhanh
- multi-stage build
- slim base image
- .dockerignore đầy đủ

Lưu ý: `python:3.11` = 1.0GB. `python:3.11-slim` = 150MB. Chỉ đổi base image đã giảm 85% size. Cẩn thận với `torch` — một dòng trong requirements có thể kéo về 2.5GB CUDA; dùng bản CPU nếu agent chỉ gọi LLM API.

Container Registry — Nơi Image Sống

Hạng mục	Google Cloud	AWS	Azure
Registry riêng tư	Artifact Registry	Elastic Container Registry (ECR)	Container Registry (ACR)
Quét lỗ hổng	Artifact Analysis	ECR image scanning	Microsoft Defender for Containers
Ai được pull	IAM policy	IAM policy	Entra ID + RBAC
Phương án trung lập	Docker Hub (public, có rate limit) · GitHub Container Registry (ghcr.io, miễn phí cho repo public) — đủ cho lab		

Image phải nằm ở registry mà platform deploy đọc được — push lên đâu là quyết định gắn liền với chọn cloud.

Container Security — Bốn Việc Làm Ngay

- **Đừng chạy bằng root.** Một dòng `USER agent` chặn cả một lớp leo thang đặc quyền nếu code bị khai thác.
- **Pin phiên bản.** `python:3.11-slim` chứ không `python:latest`; `qdrant:v1.12.1` chứ không `qdrant:latest`. Build phải tái lập được.
- **Quét image.** `docker scout cves` hoặc Trivy trong CI — chặn merge nếu có CVE mức HIGH chưa vá.
- **Đừng bake secret vào image.** Một `ENV OPENAI_API_KEY=sk-...` nằm vĩnh viễn trong layer, **ai pull được image là đọc được key**.

Lưu ý: `docker history <image>` in ra mọi lệnh đã chạy khi build. Thử trên image của bạn — nếu thấy secret ở đó thì nó cũng nằm trong registry.

Cloud Deployment Options

04

Không có 1 platform đúng cho mọi trường hợp — chọn platform dựa trên traffic, cost, control, và compliance

3 Tier Deployment



Cho khoá học

Bắt đầu Tier 1 (Railway/Render). Hiểu flow deploy trước, migrate lên Tier 2/3 khi business cần.

Phổ Compute: Càng Sang Phải Càng Ít Việc Vận Hành



Vì sao PaaS container là điểm ngọt

Agent cần **thư viện Python nặng**, **thời gian chạy dài** (LLM trả lời 10–60s), và **không cold start**. Function bị giới hạn cả ba; VM bắt bạn tự vận hành. Container PaaS là chỗ duy nhất không phải đánh đổi.

So Sánh Platform

	Railway	Render	Cloud Run	Lambda
Deploy time	< 5 phút	< 10 phút	10–15 phút	15–20 phút
Pricing	Theo mức dùng	Có free tier	Theo request	Theo lần gọi
Scale-to-zero	Không mặc định	Có (free tier)	Có	Luôn luôn
Cold start	Không	30–60s ở free tier	2–10s	1–10s
Max runtime	Không giới hạn	Không giới hạn	60 phút	15 phút
Complexity	Thấp	Thấp	Trung bình	Trung bình
Best for	MVP, demo	Side project	Production	Job ngắn, ít traffic

Lưu ý: Con số cold start thay đổi theo kích thước image và runtime — hãy **tự đo** trên chính agent của bạn thay vì tin bảng này.

PaaS Container Của Ba Ông Lớn

Hạng mục	Google Cloud	AWS	Azure
Tên dịch vụ	Cloud Run	App Runner · ECS Fargate	Container Apps
Scale to zero	Có	App Runner: không · Fargate: không	Có
Đơn vị tính tiền	vCPU-giây + request	vCPU-giờ + RAM-giờ	vCPU-giây + request
Concurrency mỗi instance	Tới 1000 / instance	1 request cho Lambda; nhiều cho Fargate	Cấu hình được
Hợp với agent vì	Rẻ nhất khi traffic thất thường	Tích hợp sẵn ALB, IAM	Gắn liền Azure OpenAI

Cả ba đều nhận thẳng một container image và tự lo TLS, autoscaling, rolling deploy — khác biệt chủ yếu là giá và hệ sinh thái quanh nó.

Railway — Deploy Trong 5 Phút

VINUNIVERSITY

Các bước

1. Kết nối GitHub repo
2. Railway tự detect Dockerfile
3. Khai báo environment variables
4. Bấm Deploy
5. Nhận public URL

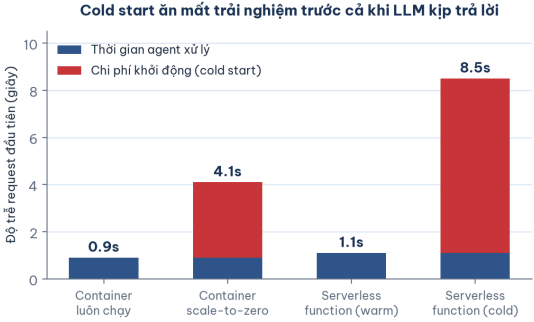
Tại sao chọn cho lab

- Không cần học IAM để deploy
- Custom domain + HTTPS tự động
- Logs real-time trong dashboard

Tip

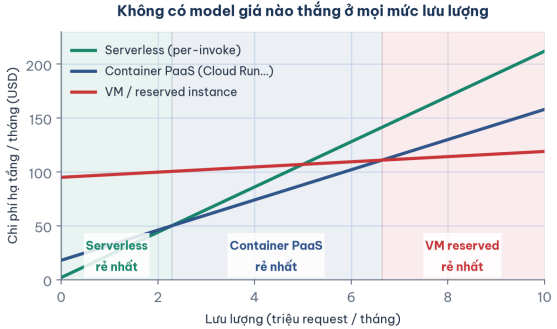
Railway inject biến `PORT` tự động — agent phải đọc `os.environ["PORT"]` và listen đúng port đó, **không hardcode 8000**. Đây là lỗi deploy phổ biến nhất của lab này.

Serverless Và Cái Giá Của Cold Start



Cold start là thời gian platform phải tải image, khởi động runtime và nạp dependency *trước khi* dòng code đầu tiên của bạn chạy. Với agent thì nó cộng dồn vào một độ trễ vốn đã dài: LLM còn cần vài giây nữa để trả lời. **Giảm nó bằng:** image nhỏ, lazy-load model, **min instances** = **1** (Cloud Run) hoặc provisioned concurrency (Lambda). Cả hai cách cuối đều đánh đổi: bạn trả tiền cho một instance luôn chạy — tức là không còn thật sự serverless nữa.

Chi Phí Đi Theo Lưu Lượng, Không Theo Sở Thích



Đường minh hoạ hình dạng chi phí, không phải báo giá — chưa tính token LLM, thường là khoản lớn nhất.

Ba model giá cắt nhau hai lần. **Ít traffic:** serverless gần như miễn phí vì không gọi thì không trả. **Traffic đều:** container PaaS rẻ hơn vì một instance phục vụ hàng trăm request. **Traffic lớn và ổn định:** VM đặt trước (reserved / savings plan) thắng vì bạn cam kết dài hạn để đổi lấy chiết khấu 40–70%. **Cảnh báo:** với AI agent, hoá đơn hạ tầng thường *nhỏ hơn* hoá đơn token — tối ưu prompt trước khi tối ưu instance.

Khi Agent Cần GPU

Khi nào thật sự cần

- Self-host model mã nguồn mở (Llama, Qwen)
- Chạy embedding model cục bộ, khối lượng lớn
- Fine-tune hoặc train
- Dữ liệu không được rời hạ tầng



HF Inference



Vertex AI

Khi nào không cần

- Agent chỉ **gọi** LLM API → GPU ở phía nhà cung cấp
- Embedding vài nghìn document → CPU vẫn kịp
- Traffic thấp → GPU nhàn rỗi vẫn tính tiền theo giờ



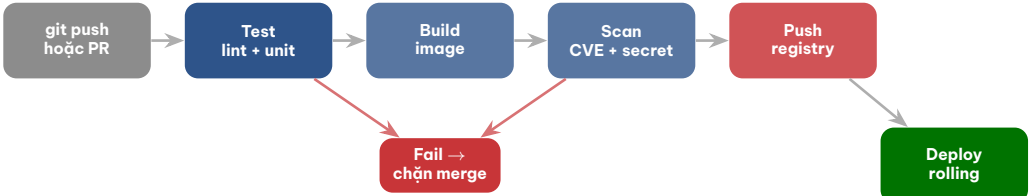
SageMaker



Azure ML

Lưu ý: Một GPU A100 tốn vài USD mỗi giờ và **tính tiền cả khi không có request nào**. Với agent của lớp này, gọi LLM API gần như luôn rẻ hơn tự host.

CI/CD — Deploy Tự Động Từ Git Push



Mỗi bước đều **tự động** và **lặp lại được** — không ai deploy bằng tay lúc 11 giờ đêm nữa.

GitHub Actions | Docker build | Terraform | Railway deploy

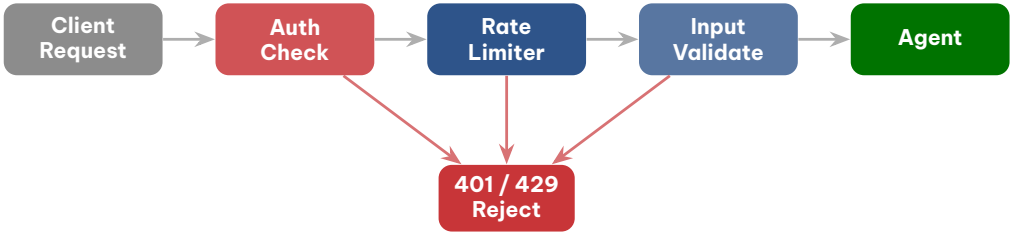
Bắt đầu nhỏ

Railway và Render đã làm sẵn phần "push là deploy". Thêm GitHub Actions khi bạn muốn **chặn** một commit hỏng trước khi nó lên production — đó mới là giá trị thật của CI.

API Gateway & Security

Agent trên cloud cần có lớp bảo vệ trước khi request đến được logic — authentication, rate limiting, và cost protection

API Gateway Architecture



Nguyên tắc

Mỗi request phải qua **auth** → **rate limit** → **validate** trước khi agent xử lý. Reject sớm = tiết kiệm tokens. Một request bị chặn ở tầng gateway tốn **0 đồng tiền LLM**; cùng request đó lọt vào agent tốn vài nghìn token.

Gateway Quản Trị Của Ba Ông Lớn

Hạng mục	Google Cloud	AWS	Azure
API gateway	API Gateway · Apigee	API Gateway	API Management
Xác thực gắn sẵn	IAM · Identity-Aware Proxy	IAM · Cognito · Lambda authorizer	Entra ID · subscription key
Rate limit / quota	Quota theo consumer	Usage plan + API key	Policy rate-limit
WAF phía trước	Cloud Armor	AWS WAF	Azure WAF
Chi phí	Theo triệu request	Theo triệu request	Theo bậc + request

Ở Tier 1 (Railway/Render) bạn tự viết ba tầng này trong FastAPI middleware — cũng ổn, chỉ là không có dashboard sẵn.

Authentication Patterns

VINUNIVERSITY

API Key

Đơn giản nhất.

Header: `X-API-Key`

Không hết hạn, thu hồi bằng cách xoá.

Dùng khi: internal, MVP, B2B

JWT Token

Stateless auth.

Bearer token + expiry

Server không cần tra DB mỗi request.

Dùng khi: user-facing app

OAuth2 / OIDC

Delegated auth.

Đăng nhập bằng Google, Entra ID

Bạn không giữ mật khẩu của ai cả.

Dùng khi: platform, SaaS

Lưu ý: Cho MVP: **API key** là đủ. Đừng over-engineer auth trước khi có user thật. Nhưng dù dùng cách nào, hãy gắn request với **một danh tính** — nếu không bạn sẽ không rate-limit được ai và không biết ai đốt tiền.

IAM — Least Privilege Trong Thực Tế

VINUNIVERSITY

Cách nhanh (và nguy hiểm)

- Gắn `AdministratorAccess` cho xong việc
- Một access key cá nhân dùng cho cả CI và production
- Key nằm trong `.env` và cũng nằm trong Slack

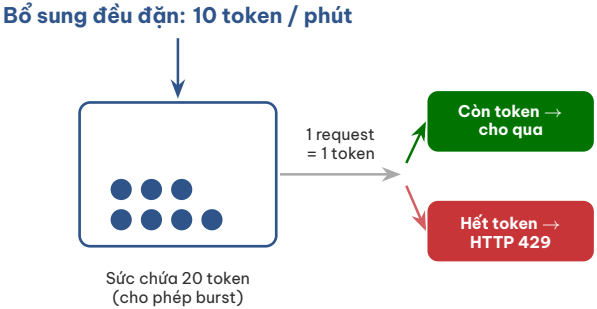
Cách đúng

- Một **role riêng cho agent**, chỉ đọc đúng bucket nó cần
- **Workload identity** thay cho long-lived key
- Bật audit log: ai gọi API nào, lúc nào

Câu hỏi kiểm tra

Nếu container agent bị chiếm quyền ngay bây giờ, **kẻ tấn công đọc và xoá được những gì?** Không trả lời được nghĩa là quyền đang quá rộng.

Rate Limiting — Token Bucket Hoạt Động Thế Nào



Vì sao không dùng bộ đếm phẳng: bộ đếm "60 request/phút" cho phép 60 request dồn vào giây đầu rồi chết cả phút. Token bucket cho burst ngắn nhưng vẫn chặn được tốc độ trung bình.

Với AI agent hãy giới hạn theo token, không chỉ theo request

Một request 200 token và một request 100.000 token tốn tiền khác nhau **500 lần**. Đếm cả hai: `requests/phút` *và* `tokens/ngày` cho mỗi người dùng.

Cost Protection — Đừng Để Agent Đốt Hết Tiền

VINUNIVERSITY

Rủi ro

- Một người dùng spam 10.000 request → hoá đơn khổng lồ
- Prompt injection kích hoạt vòng lặp → token bùng nổ
- Bug trong code gọi LLM lặp vô hạn
- Hacker dùng API key bị lộ

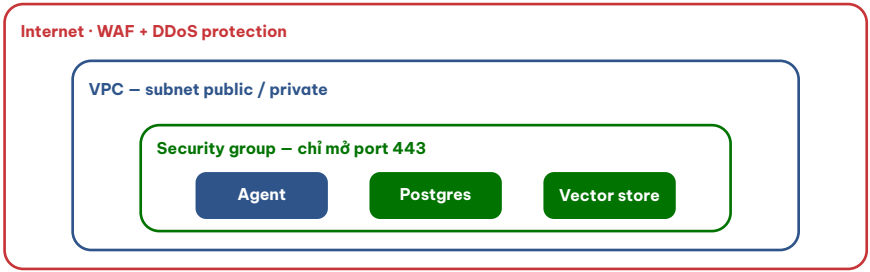
Bảo vệ

- **Rate limiting:** số request tối đa mỗi người dùng mỗi phút
- **Spending limits:** trần ngân sách theo ngày / theo tháng
- **Circuit breaker:** tự tắt khi phát hiện bất thường
- **Max tokens:** chặn cứng độ dài mỗi lời gọi

Bài học đắt giá

Nhiều startup phá sản vì quên rate limit. **Spending cap** nên là thứ được cấu hình đầu tiên — AWS Budgets, GCP Budget Alerts và Azure Cost Alerts đều **miễn phí**.

Network Security — Ba Lớp Quanh Container



Quy tắc: chỉ **load balancer** nằm ở subnet public. Agent, database và vector store nằm ở subnet private — **không có IP công cộng nào trỏ thẳng vào chúng**.

Database nhận kết nối **chỉ từ security group của agent**, không phải từ `0.0.0.0/0`. Đây là khác biệt giữa "có mật khẩu" và "không ai gõ cửa được".

- **HTTPS everywhere:** Railway/Render cung cấp SSL miễn phí tự động
- **CORS:** chỉ cho phép domain bạn kiểm soát
- **Input validation:** giới hạn kích thước, kiểm tra content type
- **Secrets:** env vars, tuyệt đối KHÔNG hardcode
- **Headers:** bỏ server version, thêm security headers
- **Log sạch:** không ghi prompt chứa dữ liệu cá nhân ra log tập trung

Lưu ý: Kiểm tra ngay: .env file có trong .gitignore chưa? API key có bị commit lên GitHub chưa? Nếu có, **revoke và tạo key mới ngay lập tức** — xoá commit không đủ, bot quét GitHub trong vòng vài giây.

Scaling & Reliability

06

Agent MVP không cần Kubernetes — nhưng cần hiểu cơ bản về scaling và reliability để hệ thống không chết khi có nhiều user hơn

Vertical vs Horizontal Scaling

VINUNIVERSITY

Vertical — máy to hơn

- Đổi 2 vCPU → 8 vCPU, xong
- Không cần đổi code
- **Trần cứng:** hết size máy là hết cách
- **Vẫn là một điểm chết duy nhất**
- Đổi size thường phải restart

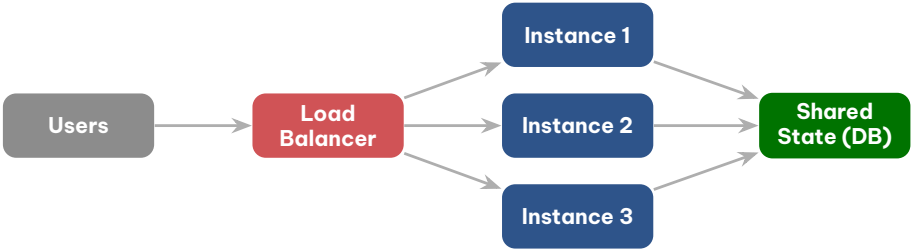
Horizontal — nhiều máy hơn

- Thêm instance sau load balancer
- Gần như không có trần
- Một instance chết, hệ thống vẫn sống
- **Điều kiện: agent phải stateless**
- Là cách mọi PaaS tự động làm cho bạn

Thực tế

Vertical trước vì nó rẻ về công sức — nhưng chỉ tới một mức. Horizontal là con đường duy nhất đi xa, và **giá vé vào cửa là stateless design**, thứ bạn phải quyết định từ lúc viết code chứ không phải lúc scale.

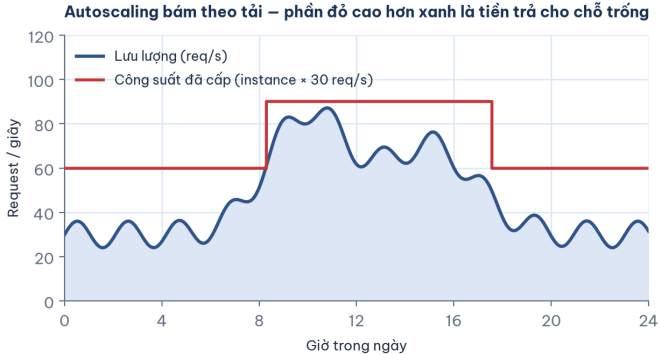
Horizontal Scaling



Điều kiện tiên quyết

Agent phải **stateless**: không giữ session trong bộ nhớ. State lưu ở DB/Redis để bất kỳ instance nào cũng phục vụ được request. Dấu hiệu bạn đã vi phạm: lịch sử chat lưu trong một `dict` toàn cục, hoặc file upload ghi vào ổ đĩa cục bộ của container.

Autoscaling — Bao Nhiêu Instance Là Đủ?



Platform thêm instance khi một **metric** vượt ngưỡng. Chọn đúng metric là cả vấn đề: **CPU** là mặc định nhưng agent chờ LLM thì CPU gần như bằng 0 dù đã quá tải. Với AI agent hãy scale theo **số request đang xử lý đồng thời** (concurrency) hoặc **độ dài hàng đợi**. Cấu hình `min instances` để tránh cold start giờ cao điểm, `max instances` để một đợt tấn công không nhân hoá đơn lên trăm lần.

Health Checks — Tự Động Phát Hiện & Khôi Phục

Liveness Probe

"Agent có còn sống không?"
Endpoint: `GET /health`
Fail → **auto-restart** instance

Readiness Probe

"Agent sẵn sàng nhận request chưa?"
Check: DB connected, model loaded
Fail → **tạm ngưng** route traffic

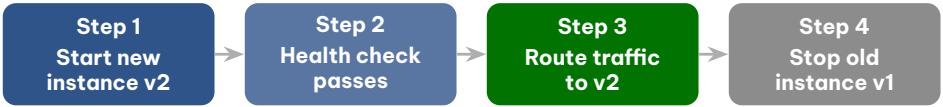
Health endpoint mẫu

`GET /health` trả về:

- status: "ok" / "degraded"
- uptime: seconds
- version: git SHA
- dependencies: DB, vector store

Lưu ý: Đừng để `/health` gọi LLM API — nó chạy vài giây một lần và sẽ tự sinh hoá đơn token.

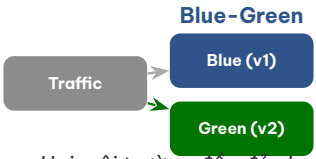
Rolling Deployment — Zero Downtime



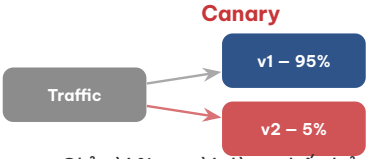
Kết quả

User không bao giờ thấy downtime. Instance mới chỉ nhận traffic **sau khi health check pass**. Railway và Render hỗ trợ sẵn — nhưng chỉ đúng nếu `/health` của bạn thật sự kiểm tra dependency, chứ không phải trả về `200` vô điều kiện.

Blue-Green Và Canary — Khi Rolling Chưa Đủ



Hai môi trường đầy đủ chạy song song.
Đổi 100% traffic trong một nhịp.
Rollback = bật lại Blue, vài giây.
Đổi lại: trả tiền gấp đôi lúc chuyển.



Chỉ vài % người dùng thấy bản mới.
Theo dõi error rate và latency,
rồi tăng dần 5% → 25% → 100%.
Lỗi chỉ ảnh hưởng một nhóm nhỏ.

Với AI agent, canary đặc biệt hợp

Đổi prompt hay đổi model là thay đổi **không thể đoán trước bằng unit test**. Cho 5% traffic chạy bản mới và so **chất lượng câu trả lời**, độ trễ, chi phí token — đó là cách duy nhất biết bản mới có thật sự tốt hơn.

SLO, SLA Và Error Budget

- **SLI** — con số bạn đo được: tỉ lệ request thành công, độ trễ p95.
- **SLO** — mục tiêu bạn tự đặt: "99,5% request thành công trong 30 ngày".
- **SLA** — cam kết *có ràng buộc hợp đồng* với khách hàng. Luôn lỏng hơn SLO.
- **Error budget** — phần được phép hỏng. 99,5% trong 30 ngày = **3 giờ 39 phút** downtime. Đó là ngân sách, không phải thất bại.

SLO	Downtime / tháng	Cái giá phải trả
99%	7 giờ 18 phút	Một instance, deploy giờ hành chính
99,9%	43 phút	Nhiều AZ, health check, rollback tự động
99,99%	4 phút 23 giây	Multi-region, on-call 24/7, tốn gấp nhiều lần

Lưu ý: Mỗi "số 9" tăng thêm khiến chi phí tăng theo bậc. Agent nội bộ dùng trong giờ hành chính **không cần 99,99%**.

- **Database snapshots:** backup vector store và user data định kỳ
- **Config versioning:** Dockerfile, docker-compose, env vars đều trong Git
- **Rollback plan:** nếu deploy mới bị lỗi → revert về version trước trong < 2 phút
- **Disaster recovery:** ghi rõ các bước khôi phục hệ thống từ backup
- **RPO / RTO:** chấp nhận mất bao nhiêu dữ liệu, và khôi phục trong bao lâu — trả lời hai câu này *trước* khi chọn tần suất backup

Lưu ý: Backup chưa test = chưa có backup. Ít nhất 1 lần phải **thử restore từ backup** để chắc chắn nó hoạt động — và bấm giờ xem mất bao lâu.

Đòn bẩy lớn nhất, theo thứ tự

1. **Token LLM** — thường là khoản lớn nhất. Prompt ngắn hơn, cache, model nhỏ hơn cho việc dễ.
2. **Instance nhàn rỗi** — scale-to-zero cho staging.
3. **Egress** — dữ liệu ra khỏi cloud tính tiền; giữ mọi thứ cùng region.
4. **Storage quên xoá** — snapshot cũ, log giữ vĩnh viễn.

Công cụ có sẵn, miễn phí

- **Budget alert** — cả ba cloud, bật trong 2 phút
- **Tag / label** mọi resource theo dự án, để biết tiền đi đâu
- **Committed use / savings plan** — giảm 40–70% nếu tải ổn định
- **Spot / preemptible** — rẻ tới 90% cho job chịu được bị ngắt

Thói quen

Xem hoá đơn **hằng tuần**, không phải cuối tháng. Bất ngờ về chi phí gần như luôn là thứ chạy âm thầm suốt 3 tuần mà không ai nhìn.

07

Hands-on & Key Takeaways

Mục tiêu cuối cùng rất cụ thể: agent có public URL, ai cũng truy cập được, có health check, có basic auth

Lab 12: Containerize & Deploy

VINUNIVERSITY

Mục tiêu lab

Đóng gói agent thành container, deploy lên cloud, và có public URL hoạt động.

1. Viết Dockerfile cho agent + dependencies (multi-stage, non-root user)
2. Build và test container trên máy: `docker build` → `docker run`
3. Viết `docker-compose.yml` cho agent + vector store
4. Deploy lên Railway hoặc Render: kết nối repo → khai báo env vars → deploy
5. Dựng health check endpoint: `GET /health` trả về status + dependencies
6. Thêm basic authentication: API key header + rate limit theo key
7. Demo: gửi request tới public URL, nhận response từ agent

Blueprint Cần Nộp

VINUNIVERSITY

Container

- Dockerfile (multi-stage, < 500MB)
- docker-compose.yml
- .dockerignore
- Health check endpoint

Deployment

- Public URL hoạt động
- Env vars đúng cách (không hardcode)
- Basic auth (API key)
- Demo request/response

Lưu ý: Không cần enterprise-grade. Điều cần chứng minh là bạn **biết cách đưa agent từ localhost lên cloud** và nó hoạt động.

Checklist Trước Khi Go-Live

- ☐ Secret nằm ở env vars, không trong Git
- ☐ `.env` có trong `.gitignore`
- ☐ HTTPS bật, HTTP redirect sang HTTPS
- ☐ `/health` kiểm tra cả dependency
- ☐ Rate limit theo người dùng đã bật
- ☐ Budget alert đã cấu hình
- ☐ Container chạy bằng user không phải root
- ☐ Image đã pin phiên bản, đã quét CVE
- ☐ Log ra stdout, không ghi dữ liệu cá nhân
- ☐ Biết cách rollback và đã thử một lần
- ☐ Backup đã chạy và đã test restore
- ☐ Có người biết phải gọi ai khi hỏng

Hoạt Động Cuối Buổi: Kiến Trúc Cloud-Native Cho Vlearn

VINUNIVERSITY

HOẠT ĐỘNG NHÓM — **60 phút · mỗi cụm bàn là một nhóm**

Đề bài

- Thiết kế kiến trúc cho **Vlearn**, **100% cloud-native**
- Không VM tự quản, không service chạy trên máy ai cả
- Managed service cho mọi tầng: compute, storage, database, auth
- Stateless · auto-scale · health check

Mỗi nhóm nộp 1 slide

- 1 sơ đồ kiến trúc
- **Gọi đúng tên dịch vụ** trên sơ đồ
- 1 câu: vì sao chọn cloud đó

Nhịp 60 phút

25' nghiên cứu · **15'** dựng slide · **20'** thuyết trình (3 phút/nhóm). Bảy bảng đối chiếu ở chương 1 là tài liệu tra cứu.

Tổng kết — Key Takeaways

VINUNIVERSITY

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. **Ba cloud, một bản đồ** — AWS, GCP, Azure đặt tên khác nhau cho cùng những tầng: compute, storage, networking, IAM, database, data platform, AI.
2. **Container = "ship anywhere"** — multi-stage build, non-root user, pin version, image < 500MB.
3. **Bắt đầu đơn giản:** Railway / Render cho MVP. Container PaaS là điểm ngọt cho agent; lên ba ông lớn khi nhu cầu thật sự đòi hỏi.
4. **Security & reliability:** least privilege, secrets manager, rate limit, spending cap — và stateless design để agent scale ngang được.

Bài tiếp theo

Monitoring, Logging & Observability

"Agent deploy xong, 3 ngày sau: latency tăng gấp đôi, cost tăng 300%. Bạn không biết cho đến khi user phàn nàn."

Bài tập về nhà

- Đọc trước: LangSmith hoặc Langfuse quickstart (20 phút)
- Chuẩn bị: agent deployed từ Lab 12 cần có endpoint để gắn monitoring
- Suy nghĩ: metrics nào quan trọng nhất cho AI agent trên production?

Tài Liệu Tham Khảo

1. Docker Official, *Getting Started Guide* — docs.docker.com/get-started/. Container basics, Dockerfile, docker-compose.
2. Railway Documentation — docs.railway.com. Deploy container lên cloud trong < 10 phút, free tier.
3. Adam Wiggins (2011), *The Twelve-Factor App* — 12factor.net. 12 nguyên tắc deploy app hiện đại.
4. Google Cloud, *GCP ↔ AWS ↔ Azure service comparison* — cloud.google.com/docs/get-started/aws-azure-gcp-service-comparison. Bảng đối chiếu chính thức, cập nhật thường xuyên.
5. Google SRE, *Site Reliability Engineering* — sre.google/books/. Chương SLO và error budget.
6. Synergy Research Group qua Statista, *Worldwide market share of leading cloud infrastructure service providers*, Q1 2026.

Hỏi & Đáp

Từ hôm nay, agent không còn chỉ chạy trên máy bạn. Nó đã là một service thật sự.